

WHITE PAPER

CHECKED OUT

Novel Encryption

Turning books into keys, and messages into lost chapters

Jeff Pittman and Ruption AI

Version 1.0 (draft) · October 2026 · novelencryption.com · github.com/RuptionAI/novel-encryption

Abstract

Books are read and then shelved. Novel Encryption gives a book a second job: it becomes the source of keys that protect real data. A person chooses a novel, such as *Moby-Dick* or *War and Peace*, and receives a key that reads like a passage from that book. Every three words in a row appear together somewhere in the novel:

Denisov came into the passage of the commander in chief. Malasha, who had settled into three armies. First the vicomte to ask so naive a question for a moment on a low voice, evidently an infantry officer who said he hoped to find out everything very thoroughly and accurately as every German has to.

*(An example 135-bit key from *War and Peace*. Never use a key that has been published.)*

To unlock the data, the person supplies the same novel and the same key. The encrypted message itself can be written as a **lost chapter**: a new passage in the book's voice whose word choices carry the message, a kind of fan fiction that only the key can read.

We are careful about what the book does and does not do. Classic book ciphers made the book the secret, and history shows that this fails. Novel Encryption treats the book as public. The book provides the words, a fingerprint that binds every key and message to it, and a shared, delightful context. The secret is the randomly drawn passage, whose strength we measure exactly in bits (128 by default, 256 for long-term secrets). The data is protected by two standard, well-analyzed algorithms, Argon2id and XChaCha20-Poly1305, rather than by word substitution. For people who would rather memorize their key, a shorter **letter-chain** style links words by their letters instead (*doubt* → *talisman* → *nobly* → *yea* → ...). We give the construction, a byte-exact specification, measurements over a thirteen-book public-domain catalog (twelve novels and the King James Bible), a security analysis, and an open-source implementation for the command line and the browser.

Contents

1. Introduction	4
2. Background	4
2.1 Book ciphers	4
2.2 Kerckhoffs's principle	5
2.3 Word-based keys	5
2.4 Modern primitives	5
3. The naive construction, and why it fails	5
4. Design goals	6
5. Construction	6
5.1 Reading the book	6
5.2 The book's fingerprint	7
5.3 The key vocabulary and the chain graph	7
5.4 Generating a key	7
5.5 Measuring strength exactly	8
5.6 Sealing data	8
5.7 Lost chapters and novel armor: writing ciphertext in the book's words	9
5.8 Wallet backups	10
6. Measurements	10
6.1 Key space	10
6.2 Narrative keys and lost chapters	11
6.3 The letter chain: cost and benefit	12
6.4 Overheads	12
7. Security analysis	13
7.1 Threat model	13
7.2 Guessing the key, today and with quantum computers	13
7.3 What the book contributes	14
7.4 Argon2id	14
7.5 Authenticated encryption	14
7.6 Lost chapters and novel armor are not steganography	15
7.7 Human factors	15
7.8 Implementation	15
7.9 Out of scope	15
8. Comparison	15
9. Applications: new life for books	16
10. Implementation and availability	16
11. Limitations and future work	17
12. Conclusion	17
References	18

1. Introduction

Physical books are being borrowed less. Libraries have responded by becoming makerspaces, classrooms and community centers. The book itself, however, still has one function: it is read. This paper asks whether a book can be *useful* in a new way and still be loved for being a book.

Our answer is to make a novel the foundation of a personal encryption key. Anyone who has a copy of the same book (and in a public-domain library, everyone does) can generate keys from it, share encrypted notes with a book club, protect a diary, or leave a sealed message in the language of Melville or Austen. The book becomes a shared, recognizable *key ring* that people remember and talk about. Like a library card, it is common to all patrons but unlocks something personal to each.

Doing this responsibly requires care. The obvious way to “encrypt with a book” is centuries old and insecure (§3). The design we present keeps everything delightful about the idea and places the actual protection in modern, standardized cryptography.

Contributions.

1. An analysis of the naive book-word substitution scheme, showing why it provides no confidentiality (§3).
2. Two key-generation methods with exact strength accounting and a stopping rule that guarantees a minimum strength for every key: **narrative keys**, passages walked through the book’s own word sequences, and **letter chains**, words linked by their letters (§5.4–5.5).
3. A **novel fingerprint** that binds keys and ciphertexts to a specific book’s vocabulary while tolerating formatting differences (§5.2).
4. **Lost chapters** (story armor), which write ciphertext as a new passage in the book’s voice, and **novel armor**, which writes it using the book’s most common words (§5.7).
5. Measurements over thirteen public-domain books: key lengths for both styles, how many typos the chain rule catches, and the size of lost chapters (§6).
6. A byte-exact specification, conformance vectors, and an open-source Rust implementation with a command-line tool and a WebAssembly build that runs entirely in the browser (§10).

2. Background

2.1 Book ciphers

Encrypting with a book is a long tradition. In 1779–1780 Benedict Arnold and Major John André exchanged messages encoded as page, line and word references into an agreed edition of Blackstone’s *Commentaries* and, later, Bailey’s dictionary. The Beale ciphers (published 1885) use the Declaration of Independence as the key text for one of their three sheets. Ottendorf ciphers use page-line-word coordinates. In each case, **the book is the key**, and secrecy depends on the adversary not knowing which book (or edition) is in use.

2.2 Kerckhoffs's principle

In 1883 Auguste Kerckhoffs argued that a cipher must remain secure even if everything about it except the key is known to the enemy. Claude Shannon later restated this as "the enemy knows the system." A published catalog of novels is part of the system, so a design in which the choice of novel is the secret fails this test immediately.

2.3 Word-based keys

Modern practice favors keys made of words because people remember and transcribe words better than random characters. **Diceware** (Reinhold, 1995) draws words uniformly from a 7,776-word list using dice, giving 12.9 bits per word. The **EFF long word list** (2016) refines the list for memorability. **BIP-39** (2013) encodes wallet seeds as words from a 2,048-word list (11 bits per word) and appends a checksum. Novel Encryption belongs to this family. The difference is that the word list comes from a book the user chooses, and the chain rule provides a structure and an error check.

2.4 Modern primitives

Argon2 won the Password Hashing Competition in 2015 and is standardized in RFC 9106. Its Argon2id variant is a memory-hard key-derivation function: deriving a key deliberately costs time and memory, which raises the price of guessing. **XChaCha20-Poly1305** is an authenticated encryption scheme. It combines the ChaCha20 stream cipher (Bernstein, 2008; RFC 8439) with the Poly1305 authenticator, extended to a 192-bit nonce so that random nonces are safe. It provides confidentiality and detects any tampering. Both are widely deployed, for example in libsodium.

3. The naive construction, and why it fails

The idea that started this project was:

Parse a novel into words. Assign each letter of the message a randomly chosen word from the novel that starts with that letter, optionally chaining further words from the last letter. The table of chosen words is the key; the novel plus the key unlocks the data.

Consider the message HELLO with *Moby-Dick*:

plaintext	H	E	L	L	O
ciphertext	harpoon	ever	leviathan	long	ocean

The first letters of the ciphertext spell the message. **No key is needed to read it.** Chaining (harpoon → nantucket → ...) does not help, because the first word of each chain still begins with the plaintext letter.

Suppose we drop the first-letter rule and let each letter map to arbitrary words. The result is a **homophonic substitution cipher**: each letter has several interchangeable "homophones." Such ciphers were broken routinely from the Renaissance onward. Word boundaries, letter-pair statistics and any known fragment of plaintext

(a greeting, a name) quickly expose the mapping. Making the novel secret does not rescue it either: with a public catalog of novels, an attacker simply tries every one, and even an unpublished book adds only a modest number of possibilities compared with a random key.

Two lessons follow, and they shape everything else in this paper:

1. **The book cannot be the secret.** It must be treated as public.
2. **Substitution cannot be the cipher.** Confidentiality must come from a modern, analyzed algorithm.

Several parts of the original idea are worth keeping: the book as the source of the words, keys made from the book's language, the chaining of words by their letters, the ability to choose how deep the chain goes, and the requirement that both the book and the key are needed to unlock the data. The construction below keeps all of them.

4. Design goals

- **G1 Book-centric.** The chosen book determines the vocabulary, the key, and how the output looks. Without the same book, nothing opens.
- **G2 Standard security.** Confidentiality and integrity rest only on Argon2id and XChaCha20-Poly1305, under a key with measured strength.
- **G3 Measurable strength.** Every key carries an exact strength in bits, and the default guarantees at least 128.
- **G4 Human-friendly keys.** Keys are real words that people can say aloud, write on a bookmark, and type with useful error messages.
- **G5 Robust binding.** The same book from a different source (re-wrapped lines, different punctuation) still works, but a different book does not.
- **G6 Open and local.** Open specification, open source, test vectors. Encryption runs on the user's device, never on a server.
- **G7 Delight.** Using it should feel like a love letter to books.

5. Construction

Section references (§) in `SPEC.md` give byte-exact details. This section explains them.

5.1 Reading the book

The text is normalized (Unicode NFKD, accents removed), lowercased, and split into words made of the letters a–z. Numerals and punctuation are dropped, and other characters separate words. "Café" becomes `cafe` and "Ahab's" becomes `ahab` and `s`.

5.2 The book's fingerprint

We hash (SHA-256, domain-separated) the sorted set of *distinct* words in the book. Two copies of *Moby-Dick* with different line wrapping or curly versus straight quotes produce the same fingerprint. A copy with even one changed word does not, and the mismatch is detected. The fingerprint is mixed into every key derivation, so a key used with the wrong book cannot decrypt.

5.3 The key vocabulary and the chain graph

Keys use words of 3–15 letters. Each word is a node in a graph, with an edge from word u to every word that starts with u 's last letter. Some words lead nowhere: if no eligible word in the book starts with x , a word ending in x is a dead end. We repeatedly remove such words until every remaining word has a successor. Across our catalog, the length limits and pruning together set aside about 1% of a book's distinct words (2.1% for *Alice*, whose list includes many short words).

5.4 Generating a key

All randomness comes from the operating system's cryptographic random number generator (in the browser, `crypto.getRandomValues`), with unbiased rejection sampling. The user chooses how deep the key goes: by default the generator keeps going until the key reaches 128 bits (or 256 for a long-term key), or it can produce an exact number of words. Requests that would fall below 128 bits are refused unless the user explicitly opts in.

Narrative keys (the default). We read the book as a stream of words and count every run of three consecutive words. Chapter headings and editorial insertions such as "[Illustration]" are skipped, so they are not stitched into keys.

```
(w1, w2) ← uniform choice among word pairs that begin a sentence in the book
repeat:
  wi+1 ← a word that follows (wi-1, wi) in the book, chosen with probability
           proportional to how often the book continues that pair with it
until the key is strong enough, then finish the sentence
```

Every three consecutive words of the key therefore appear together in the novel, and common continuations are favored over odd ones, so the key reads like the book. We tested three designs before settling on this one:

Context	Reads like	Words for 128 bits (<i>Moby-Dick</i>)
One previous word	word salad	~23
Two previous words	the book	~75
Three previous words	near-verbatim pages	~430

Once the target strength is reached, the generator continues to the next point where the book ends a sentence, adding at most 12 words, so keys end naturally. For display, each word takes its usual capitals from the book (*Ahab*, *Queequeg*), and between each pair of words the key shows the punctuation the book most often puts there. Display punctuation and capitals are derived from the words alone. They carry no secret and never need to be typed: `denisov came into the passage` opens the same key.

Letter chains (shorter, to memorize). Each word starts with the last letter of the word before it:

```
w1 ← uniform choice from the whole vocabulary V
repeat:
  wi+1 ← uniform choice from the words beginning with the last letter of wi
until the key is strong enough or has the requested number of words
```

A chain needs only 14–19 words for 128 bits, short enough to memorize or write on a bookmark.

Both styles feed the same key derivation, and a key is accepted if it is valid in either style.

5.5 Measuring strength exactly

Let p_i be the probability of the choice made at step i : $1/n$ for a uniform choice among n options, or *count/total* for a narrative continuation. A generated key has probability $\prod p_i$, so its **surprisal** is

$$\text{bits} = -\log_2 p_1 - \log_2 p_2 - \dots - \log_2 p_k.$$

The default stopping rule ends generation only after bits ≥ 128 . Every key it can produce therefore has probability at most 2^{-128} . That is a guarantee on the **min-entropy** of the key distribution, the strongest standard measure, and not merely an average. It holds even though narrative keys favor common continuations: a common choice simply adds fewer bits, so the key grows longer. The stopping rule depends only on the words generated so far, so the set of possible keys is prefix-free and the probabilities are well-defined.

This accounting is valid only for keys produced by the generator. A passage a person copies out of the book, or a chain they make up, passes the format checks but is far weaker than the number suggests. The tools therefore always generate keys and never ask users to invent them.

5.6 Sealing data

```
password    = "NovelEncryption/v1/key\n" || fingerprint(book)
              || key words joined by spaces
key || nonce = Argon2id(password, random 16-byte salt,
                        64 MiB, 3 passes, 1 lane) → 32 + 24 bytes
message     = format byte || salt
              || XChaCha20-Poly1305(key, nonce, flags || payload,
              AD = format byte || salt)
```

The design aims for the smallest message that gives up no security:

- **No stored nonce.** Each message has a fresh random salt, so it has its own key and nonce, both derived from Argon2id. Only the salt needs to travel.
- **One byte of settings.** The standard Argon2id cost is named by the format byte. Custom costs add 9 bytes.
- **Compression when it helps.** The data is compressed (DEFLATE) before encryption, and the compressed form is kept only if it is smaller. A flag inside the encrypted payload records the choice.

A message is therefore exactly **34 bytes** longer than its (possibly compressed) payload, whether the book is *Alice* or *War and Peace*. **The book never travels with the message.** The recipient already has it, so a book's size costs nothing. Appending the book to a message would only make the message larger, and it would add no secrecy, because the book is public.

The header is authenticated. Before deriving anything, decryption checks that the key fits the book. A mismatch is reported by word position ("word 7 doesn't appear in this novel"), while a wrong-but-plausible key, a wrong book, or a tampered message all produce the same error and reveal no plaintext.

5.7 Lost chapters and novel armor: writing ciphertext in the book's words

Encrypted data is random bytes. To carry it in a chat or an email it can be written as a **compact** single-line string (base64url, the smallest text form), as a standard text block with BEGIN/END lines, as a **lost chapter**, or as **novel armor**.

Lost chapters (story armor). The encrypted message steers a walk through the same three-word model used for narrative keys. The walk opens with one of the book's 256 most common sentence openings, which carries 8 bits. Each later word is one of the (up to) four most common ways the book continues the previous two words, which carries up to 2 bits. A continuation with only one option carries none. Reading the chapter back with the same book recovers each choice and therefore every bit. When the message runs out, the story takes the most common path to the end of a sentence. The random salt is encoded first, so no two chapters open the same way:

MOBY-DICK: A LOST CHAPTER

But when you come back to it. But what is the great Sperm whale, and the other end of it, I say, we good Presbyterian Christians should be the...

The result is a new passage in the book's voice, a kind of fan fiction, that carries a sealed message. A reader with the book and the key opens the message. Anyone else holds a strange new chapter of *Moby-Dick*. Capitals, punctuation, line breaks and the heading are ignored when decoding, so a chapter survives being pasted through email or chat.

Novel armor, the compact alternative, writes each byte as one of the book's 256 most frequent words, grouped into sentences:

Whales thou queequeg last and and the the and the his the. The the and between into other. New round pe-quod called oil though. Over seems end stood because high arm about death think whale seems hard. Great heads night their world tell came sometimes have...

Both are *encodings*. They do not make the message more secret, and they do not pretend to be real prose: a careful reader can tell a lost chapter is machine-made, and length still reveals roughly how long the message is. Their job is to make an encrypted message belong to the book, and to need that same book to decode. Even the *container* of a Moby-Dick message is written in Moby-Dick's words.

5.8 Wallet backups

Cryptocurrency wallets back up their secret as a 12–24-word recovery phrase from the BIP-39 standard, which encodes 128–256 bits of entropy. Novel Encryption can write that same entropy into a book: as a **passage** (a story encoding, like a lost chapter, with up to 3 bits per word) or as a **letter chain** (about 9 bits per word). Because both forms carry exactly the phrase's bits, the book-form backup always converts back to the original phrase, which works with every standard wallet.

Book	24-word phrase as a passage	24-word phrase as a chain
The King James Bible	~124 words	~35 words
Pride and Prejudice	~178 words	~41 words
Moby-Dick	~194 words	~34 words
Alice's Adventures in Wonderland	~272 words	~48 words

The conversion adds a 32-bit check computed from the book's fingerprint and the entropy. It serves two purposes. First, restoring with the wrong book, or after a word was changed, fails instead of producing a different wallet. Second, it guards against the **brain-wallet** failure, in which people chose a memorable phrase themselves and lost their funds to attackers who guessed it. A passage copied out of the book has about a one-in-four-billion chance of passing the check, so the tool will not turn chosen text into a wallet. The randomness always comes from the wallet's own generated phrase.

The converter ships as a command (`novelenc seed`) and as a single offline HTML file whose security policy blocks all network access. It is deliberately not offered on the website, so that people are never encouraged to type a recovery phrase into a web page.

6. Measurements

All figures come from `novelenc analyze` over the pinned catalog texts (Project Gutenberg editions, stripped of license headers).

6.1 Key space

Novel	Words in text	Key vocabulary	1st word (bits)	Each chained word (bits)	Letter-chain words for 128 bits
Alice's Adventures in Wonderland	27,427	2,521	11.30	6.66	19
Treasure Island	70,496	5,831	12.51	7.90	16
Frankenstein	75,318	6,912	12.75	8.12	16
The Adventures of Sherlock Holmes	105,855	7,741	12.92	8.22	16
Pride and Prejudice	128,560	6,653	12.70	7.96	16
A Tale of Two Cities	138,434	9,617	13.23	8.58	15
Dracula	163,374	9,179	13.16	8.52	15

Novel	Words in text	Key vocabulary	1st word (bits)	Each chained word (bits)	Letter-chain words for 128 bits
Great Expectations	189,049	10,661	13.38	8.65	15
Jane Eyre	189,419	12,431	13.60	8.96	14
Moby-Dick	219,047	16,837	14.04	9.37	14
Middlemarch	323,817	15,146	13.89	9.17	14
War and Peace	573,084	17,304	14.08	9.39	14
The King James Bible	792,180	12,490	13.61	9.10	14

"Each chained word" is the long-run expected entropy of a word after the first, computed exactly as the stationary average of a Markov chain over last letters. Richer vocabularies give shorter keys: Melville's 16,837 usable words reach 128 bits in 14 words, while *Alice* needs 19.

6.2 Narrative keys and lost chapters

Narrative key lengths are the means of 40 generated keys per book, including the words added to finish the sentence. Lost-chapter sizes are for a 100-byte sealed message (66 bytes of incompressible plaintext).

Novel	Narrative key, 128 bits	Narrative key, 256 bits	Lost chapter, words per byte
The King James Bible	44	84	4.8
War and Peace	49	99	5.0
Middlemarch	57	110	
Great Expectations	60	117	
Dracula	63	125	
Pride and Prejudice	65	131	6.1
Jane Eyre	67	136	
A Tale of Two Cities	69	141	
The Adventures of Sherlock Holmes	69	142	
Moby-Dick	75	145	6.3
Treasure Island	80	166	
Frankenstein	90	171	
Alice's Adventures in Wonderland	98	197	8.7

A narrative key averages 1.3–3.1 bits per word, against 6.7–9.4 for a letter chain. It is three to six times longer, a paragraph rather than a phrase, and it reads like the book. The longest and most varied books (the King James Bible, *War and Peace*, *Middlemarch*) give the shortest passages. Narrative keys are meant to be kept on a book-mark or in a password manager, not memorized. The letter chain remains for people who want to memorize their key.

We expect typo detection to be stricter for narrative keys than for chains, because a changed word must still form three-word sequences that occur in the book with *both* of its neighbors. We have not yet measured this exhaustively. The figures below are for the chain, where we have.

A short note such as "Meet me at the library at noon." becomes a lost chapter of about 400 words.

6.3 The letter chain: cost and benefit

Cost. The chain rule restricts each word to those starting with one letter. In *Moby-Dick* this lowers the entropy of a chained word from 14.04 bits (any word) to 9.37 bits, so keys are about 40% longer than unconstrained keys from the same book (14 words rather than 10). The cost is real, and we accept it deliberately.

Benefit 1: error detection. We applied every single-character typo (substitution, insertion, deletion, adjacent swap) to every word in each vocabulary and counted the typos that would go unnoticed:

Novel	Typos tested	Unnoticed, vocabulary check only	Unnoticed, with chain rule
Alice's Adventures in Wonderland	874,865	0.39%	0.10%
Pride and Prejudice	2,773,953	0.25%	0.07%
Moby-Dick	6,934,603	0.48%	0.16%
War and Peace	7,210,854	0.42%	0.13%
The King James Bible	4,934,630	0.42%	0.15%
All thirteen books	53.8 million	0.25–0.49%	0.07–0.16%

Most typos are already caught because the misspelling is not a word in the book. The chain rule catches about 70% of the remaining dangerous cases, where a typo turns one real word into another (*whale* → *whales*, *ever* → *never*), because such a typo usually changes the first or last letter and breaks a link. The tools then point at the exact word. Swapped words are almost always caught too, and a word dropped from the middle of a key is caught unless it begins and ends with the same letter. (A dropped first or last word leaves a valid chain; decryption then simply fails.)

Benefit 2: structure people can work with. Each word hands a letter to the next, which gives a key rhythm and something to check when reading it aloud or copying it from a bookmark: "*doubt* ends in **t**, so the next word starts with **t**: *talisman*." This is the original project's chaining idea, now working as a checksum rather than as a cipher.

6.4 Overheads

Form	Size	Notes
Binary	payload + 34 bytes	format byte, salt, flags byte, tag
Compact armor	×1.33 of binary	one base64url line
Text armor	×1.35 of binary, plus ~80 characters	BEGIN/END lines
Novel armor	×5.5–6.2 of binary	one word (about 4–5 letters plus a space) per byte
Lost chapter	5–9 words per byte of binary	a new passage; up to 8 KiB of sealed data

Measured examples (standard profile, *Moby-Dick* key):

Plaintext	Sealed
"Meet me at the library at noon." (31 bytes)	64 bytes binary · 86-character compact string

Plaintext	Sealed
<i>Alice's Adventures in Wonderland</i> (151,095 bytes)	53,327 bytes (35%, compressed)
<i>Moby-Dick</i> (1,234,509 bytes)	499,905 bytes (40%, compressed)
100,000 random bytes (incompressible)	100,034 bytes

A sealed text file is usually *smaller* than the original.

Key derivation with the default cost (64 MiB, 3 passes) plus preparing *Pride and Prejudice* took 0.13 s on an Apple-silicon Mac with the native build. Browser builds are slower but remain comfortably interactive.

7. Security analysis

7.1 Threat model

The adversary knows the full specification, the catalog, **which novel was used**, and has the encrypted message. The adversary may alter messages and may know or choose other plaintexts. The adversary does not know the key chain. We do not defend against malware on the user's device or a user who discloses their key.

7.2 Guessing the key, today and with quantum computers

With a generated 128-bit key, the adversary must search $2^{128} \approx 3.4 \times 10^{38}$ possibilities, and each guess costs a full Argon2id evaluation using 64 MiB of memory. Even at an unrealistic 10^{12} guesses per second, the expected search time exceeds 10^{19} years.

Quantum computers. Novel Encryption uses no public-key cryptography, so Shor's algorithm, the quantum attack that breaks RSA and elliptic curves, does not apply. The relevant quantum attack is Grover's search, which in principle finds an n -bit key in about $2^{(n/2)}$ steps:

Component	Classical	Against Grover
XChaCha20 key (256 bits)	256	~128 bits: ample
Poly1305, SHA-256, Argon2id output	n/a	unaffected in practice
Key chain, standard (128 bits)	128	~64 bits in theory
Key chain, long-term (256 bits)	256	~128 bits

In practice the standard key is stronger than "64 bits" suggests. NIST counts 128-bit symmetric keys (AES-128) as meeting its lowest post-quantum security category. Grover's search cannot be split efficiently across many machines. On top of that, every guess here would require running Argon2id with 64 MiB of memory inside a quantum computer.

For secrets that must stay safe for decades, against an adversary who records messages today and decrypts them later, Novel Encryption offers a **long-term key** of at least 256 bits. Only the key's length changes: the book, the format and the algorithms stay the same.

Book	Standard key (128 bits)	Long-term key (256 bits)
<i>Moby-Dick, War and Peace</i>	14 words	~27 words
<i>Pride and Prejudice</i>	16 words	~32 words
<i>Alice's Adventures in Wonderland</i>	19 words	~38 words

Books with large vocabularies keep long-term keys shortest. We do not add entropy from outside the book, such as a key file or a random number: that would break the promise of "a book and some words," and a private book's secrecy cannot be measured.

Why not more than 256 bits? The generator accepts targets up to 4,096 bits, but going past 256 adds no real security. Every message is ultimately sealed with a 256-bit XChaCha20 key, so a longer passage still funnels into 256 bits. And 256 bits is already beyond physical limits. By Landauer's principle, merely *counting* to 2^{256} on an ideal computer at the temperature of deep space would take more energy than the Sun will emit in its lifetime, before a single key is even tested [13]. NIST defines its highest post-quantum security category as being as hard as searching for a 256-bit AES key, and a 256-bit key here meets that bar. Beyond this point the risks that remain are not in the mathematics: a compromised device, a key that is shared or lost, or tampered software. Section 7.8 addresses the last of these.

7.3 What the book contributes

For catalog books the novel adds **no secrecy**, and our strength figures never count it. Its roles are as follows:

- **Vocabulary.** The book determines the key's words and its look.
- **Binding.** The fingerprint is part of the key-derivation input, so the same words used with another book derive an unrelated key, and keys are not interchangeable across books.
- **Domain separation.** Identical key words drawn from two books with overlapping vocabularies never collide.

A private or unusual text (a family memoir, say) can add some uncertainty for an attacker who does not have it, but this is hard to quantify and should be treated as a bonus, never as the basis of security.

7.4 Argon2id

For generated 128-bit keys, the memory-hard KDF is a second line of defense. It matters most when users opt into shorter chains: an 8-word *Moby-Dick* key (about 80 bits) remains very expensive to attack because each of the roughly 2^{80} guesses costs 64 MiB of memory-hard work. Decryptors enforce upper bounds on the cost parameters recorded in the header (1 GiB, 64 passes, 16 lanes), so a forged message cannot exhaust a device's memory.

7.5 Authenticated encryption

XChaCha20-Poly1305 provides confidentiality against chosen-plaintext attack and ciphertext integrity. Any change to the header (including the cost parameters) or to the ciphertext is rejected, and no plaintext is released. Its 192-bit random nonces, Every message has a fresh 128-bit salt and therefore its own derived key *and* nonce, so a nonce is never reused. A single key can safely protect many messages.

Compression makes a message's length depend on its content. This matters only when an attacker can insert text of their choosing into a message that also contains a secret, and observe the resulting sizes (the CRIME/BREACH pattern). That is unusual for personal messages and files, but tools that seal mixed content can turn compression off (`--no-compress`).

7.6 Lost chapters and novel armor are not steganography

A lost chapter is generated by a three-word model, and a careful reader or a statistical test can tell it from the author's prose. Its length grows with the message's length. Novel armor uses only 256 words with roughly equal frequency, and every message from a given book begins with the same word. Neither hides that a message exists, and neither adds secrecy. Both should be understood as playful, book-specific alternatives to base64. The protection always comes from the encryption underneath. A tampered chapter is rejected, either because a word no longer fits the book or because the authentication tag fails.

7.7 Human factors

- **Keys must be generated, not invented.** Self-chosen chains and passages copied from the book are predictable. The software only ever generates keys, and it says so.
- **Storing keys.** A narrative key is a paragraph: keep it in a password manager, or print it on a bookmark and keep it like a house key. A letter chain (about 14 words) can be memorized. Anyone who has the key and knows the book can open the data.
- **No recovery.** Lose the key and the data is gone. That is the point of encryption.

7.8 Implementation

The reference implementation is written in Rust using the RustCrypto `argon2` and `chacha20poly1305` crates. It zeroizes key material after use, validates every header field before use, and has no unsafe code of its own. The web version runs the same Rust code compiled to WebAssembly inside the visitor's browser, and no book, key or data is sent to a server. The site loads nothing from any other origin: no third-party scripts, fonts or analytics. It is served with a strict Content Security Policy that blocks inline and external code, along with HSTS, frame denial and a no-referrer policy. A web page is nonetheless only as trustworthy as the code it serves. For high stakes, use the command-line tool or verify the published build against the open source.

7.9 Out of scope

Novel Encryption is a shared-secret (symmetric) system. It does not provide public-key exchange, sender authentication between parties who share a key, or forward secrecy. It has **not yet been independently audited**. We invite review (see `SECURITY.md`).

8. Comparison

	Classic book cipher	Diceware / EFF	BIP-39	Novel Encryption
Word source	a secret book	a fixed list	a fixed list	any book the user loves

	Classic book cipher	Diceware / EFF	BIP-39	Novel Encryption
What is secret	the book	random words	random words	a randomly drawn passage or chain
Bits per word	n/a	12.9	11	1.3–3.1 (narrative), 6.7–9.4 (chain)
Words for 128 bits	n/a	10	12 (+checksum)	44–98 (narrative), 14–19 (chain)
Reads like	the book	random words	random words	the book (narrative)
Error detection	none	none	4-bit checksum (12 words)	every word checked against the book, position reported
Protects data with	the substitution itself	external tool	external tool	built-in Argon2id + XChaCha20-Poly1305
Secure under Kerckhoffs	no	yes	yes	yes

Novel Encryption's keys are longer than Diceware's for the same strength: much longer for a narrative passage, a little longer for a chain. In exchange, the key comes from a book the user chose and reads like it. The user also gets per-word error localization, binding to that book, and a complete encryption tool rather than only a key.

9. Applications: new life for books

This project began on a library board, with a question: what can a book *do* when fewer people borrow it? Some answers:

- **Library programs.** "Encrypt with a Classic" workshops in which patrons choose a book from the library's public-domain shelf, generate a key, and send each other sealed messages. These sessions can open with §3: breaking the naive cipher by hand is a wonderful first lesson in why cryptography is hard.
- **Book clubs.** A club's current book becomes its key ring for sharing private notes and puzzles.
- **Fan-fiction encryption.** Members exchange "lost chapters" of the book they are reading. Each chapter is a new passage in the author's voice, and to those holding the key, a private letter. Writing programs can invite patrons to illustrate, perform or continue the strange chapters their messages become.
- **Keepsakes.** A family recipe sealed with Grandma's favorite novel. A time capsule sealed with the book a class read that year.
- **Escape rooms and treasure hunts.** Clues hidden in a book's vocabulary. The chain rule makes a natural puzzle mechanic.
- **Teaching.** A concrete, hands-on way to teach entropy, Kerckhoffs's principle and authenticated encryption, using the books students are already reading.
- **A reason to visit.** A printed "key bookmark" with the book's title, cover and a space to write a key, handed out at the circulation desk.

10. Implementation and availability

- **Specification:** `docs/SPEC.md` (byte-exact) and `vectors/v1.json` (conformance vectors).

- **Rust library:** `novel-encryption`, the reference implementation.
- **Command-line tool:** `novelenc` (`inspect`, `keygen` with `--style narrative|chain` and `--long-term`, `check`, `encrypt` with `--armor story|novel|compact|text|binary`, `decrypt`, `seed to|from` for wallet backups, `analyze`).
- **Offline wallet-backup page:** `tools/novel-seed-offline.html`, one self-contained file.
- **Web:** the same library compiled to WebAssembly, at `novelencryption.com`. Everything runs locally in the browser.
- **Catalog:** twelve public-domain novels and the King James Bible from Project Gutenberg, pinned by fingerprint.
- **License:** MIT or Apache-2.0, at the user's option.

Planned ports reuse the Rust core: Python (PyO3), Swift and Kotlin (UniFFI), and JavaScript/TypeScript (the WebAssembly package). Independent implementations are welcome and can be checked against the vectors.

11. Limitations and future work

- **English-centric tokenization.** Version 1 keeps only the letters a–z after removing accents. Novels in other scripts need a Unicode-aware tokenizer and a per-script chain rule, planned for version 2.
- **Edition sensitivity.** The fingerprint tolerates formatting changes but not textual variants. The catalog therefore pins exact texts and never changes them.
- **Physical books.** Today the book must be in digital form. We plan an edition-pinned "page index" so that a key's words can be found, and verified, in a specific printed edition on the library shelf. This would link the physical copy to the digital key.
- **Fan fiction with a plot.** Lost chapters follow the book's voice from three words to the next, but they have no storyline. A language model could write coherent new scenes while carrying the message the same way, provided that both sides can reproduce its word probabilities exactly. That rules out hosted AI services, whose output varies, but a small, pinned model running locally would work. We plan to explore this.
- **Audit.** An independent cryptographic review is planned before version 1.0 is marked final.
- **Public-key mode.** Sharing a key in person works for book clubs. A public-key mode would let strangers exchange sealed messages under the same book.

12. Conclusion

A book cannot be a secret, but it can be a home for one. Novel Encryption keeps what is charming about book ciphers: the shared text, the book's own words and voice, and even new chapters written in that voice. It replaces what made those ciphers fail with measured randomness and modern authenticated encryption. The result gives every public-domain novel on a library's shelves a second purpose: protecting what matters to its readers, and turning their secrets into stories.

References

1. A. Kerckhoffs, "La cryptographie militaire," *Journal des sciences militaires*, 1883.
2. C. E. Shannon, "Communication Theory of Secrecy Systems," *Bell System Technical Journal* 28(4), 1949.
3. A. G. Reinhold, "The Diceware Passphrase Home Page," 1995.
4. J. Bonneau, "EFF's New Wordlists for Random Passphrases," Electronic Frontier Foundation, 2016.
5. M. Palatinus, P. Rusnak, A. Voisine, S. Bowe, "BIP-39: Mnemonic code for generating deterministic keys," 2013.
6. A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson, "Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications," RFC 9106, 2021.
7. D. J. Bernstein, "ChaCha, a variant of Salsa20," 2008.
8. Y. Nir, A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, 2018.
9. S. Arciszewski, "XChaCha: eXtended-nonce ChaCha and AEAD_XChaCha20_Poly1305," IRTF CFRG Internet-Draft.
10. D. Kahn, *The Codebreakers*, rev. ed., Scribner, 1996 (book ciphers; the Arnold–André correspondence).
11. *The Beale Papers*, Lynchburg, Virginia, 1885.
12. Project Gutenberg, <https://www.gutenberg.org> (catalog texts).
13. B. Schneier, *Applied Cryptography*, 2nd ed., Wiley, 1996, §7.1 (thermodynamic limits on brute force).

© 2026 Jeff Pittman and Ruption AI. This paper is licensed CC BY 4.0. The software is licensed MIT OR Apache-2.0.